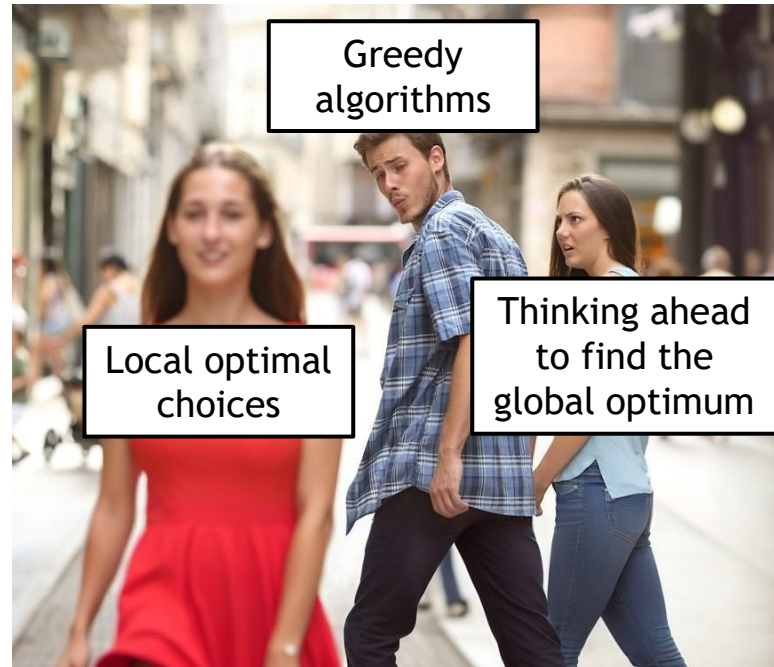
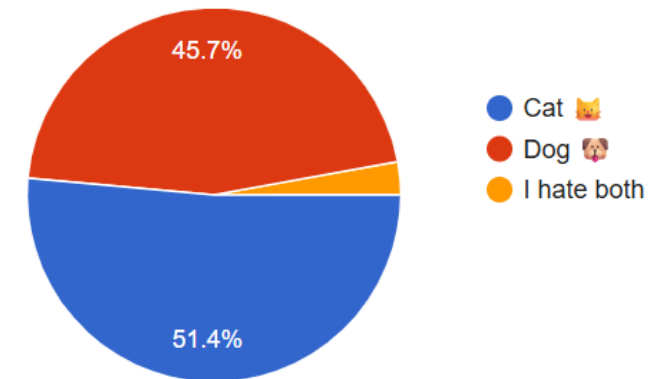


D4: Dimensionality and Greedy Algorithms



Are you a cat/dog person?

35 responses



Sec 21: Th 4:30-5:30pm 2153 GGBL
IA: Anthony Chalhoub

Agenda

- ▶ Dimensionality
- ▶ Graph DP
- ▶ Greedy Algorithms
 - ▶ Exchange Argument
 - ▶ Counterexample

Dimensionality



The Full DP Recipe

- ▶ Step 0: Determine suitable dimensionality for the problem ← Will talk about this now

// Recurrence relation- Math only, no data structures

- ▶ Step 1: Identify the value version of the problem
- ▶ Step 2: Identify the base case(s)
- ▶ Step 3: Identify subproblem dependencies

// Actual implementation- Think like a programmer

- ▶ Step 4: Describe the algorithm (see [Handout 3: Giving an Algorithm](#))
 - ▶ Step 4a: Reconstructing solution (backtracking)
 - ▶ Step 4b: Correctness Proof
 - ▶ Step 4c: Runtime analysis

Dimensionality

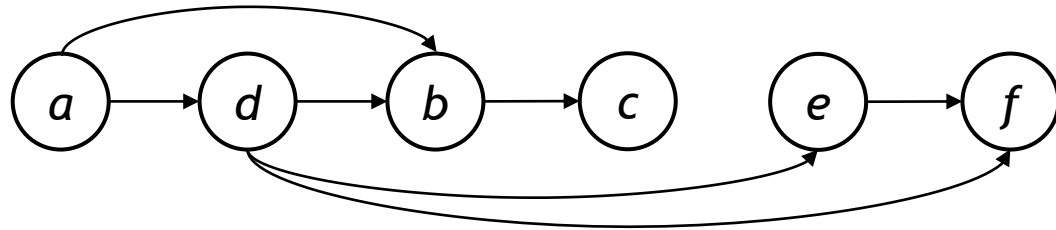
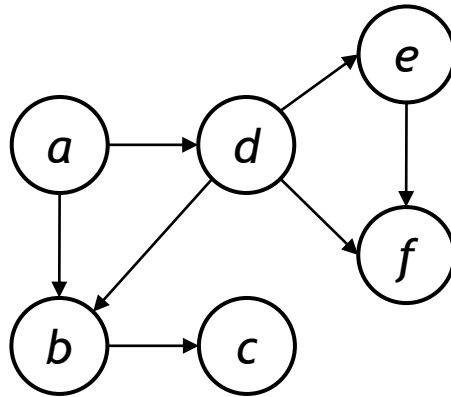
- ▶ The **FIRST** thing to think about when you get the problem!
 - ▶ Imagine spending 20 mins thinking about a 3D solution when the problem can be solved in 1D...
- ▶ Very *vaguely*, dimensionality = number of **independent variables** that you need to keep track of at each step of the problem
 - ▶ LIS (1D): Just need one to keep track of where the subsequence 'ends'
 - ▶ 0-1 Knapsack (2D): Need one for 'value' and one for 'capacity'
 - ▶ LCS (2D): Need one for each 'last index to consider' for each string
 - ▶ Floyd-Warshall (3D): One for start vertex, one for end vertex, and one "can use up to" vertex in the middle
- ▶ **Label your table:** When picturing the table, think about what each row/column correspond to

Graph Dynamic Programming



Topological Ordering

- **Definition:** *Topological ordering* for a Directed Acyclic Graph (DAG) is a **linear ordering** of vertices such that for every directed edge (u, v) , vertex u comes **before** v in the ordering



- This allows us to assign rows/columns of a table to the vertices in a DAG

Shortest-path Problems

- ▶ **Assumption:** $G = (V, E)$ is a weighted directed graph with no negative-weight cycle
- ▶ **Single-Source Shortest Path (SSSP):** Given a "source" vertex $s \in V$, find a shortest path from s to each vertex $t \in V$
 - ▶ Value version: Given s , find the shortest version $d(s, t)$ for every $t \in V$
- ▶ **All-Pairs Shortest Path (APSP):** For every pair of vertices $s, t \in V$, find a shortest path from s to t
 - ▶ Value version: Find the shortest distance $d(s, t)$ for every $s, t \in V$
- ▶ **Connection:** We can solve SSSP using each vertex as s to get a solution to APSP
- ▶ **Key observations:**
 - ▶ If a shortest path P from s to t goes through vertex v , then P must be a shortest path from s to v , followed by a shortest path from v to t (by substitution argument)
 - ▶ Since there is no negative-weight cycle in the graph, there is a path from s to t with no cycle in it
 - ▶ This means when considering a shortest path, we can assume WLOG that the path has no cycle

Bellman-Ford vs Floyd-Warshall

Bellman-Ford (SSSP)

- ▶ Let $d^{(i)}(s, v)$ = length of a shortest path from s to v that uses **exactly i edges**

- ▶ For the base cases

$$d^{(0)}(s, v) = \begin{cases} 0 & \text{if } s = v \\ \infty & \text{if } s \neq v \end{cases}$$

- ▶ For the recursive case

$$d^{(i)}(s, v) = \min_{(u,v) \in E} \{d^{(i-1)}(s, u) + w(u, v)\}$$

u that minimizes the total distance

Use $i - 1$ edges to get from s to u

Fix the last edge to be (u, v)

Floyd-Warshall (APSP)

- ▶ Let $d^{[i]}(s, v)$ = length of a shortest path from s to v whose **internal vertices** (if any) are in $\{v_1, \dots, v_i\}$

- ▶ For the base cases

$$d^{[0]}(s, t) = \begin{cases} 0 & \text{if } s = t \\ w(s, t) & \text{if } (s, t) \in E \\ \infty & \text{otherwise} \end{cases}$$

- ▶ For the recursive case

$$d^{[i]}(s, t) = \min \left\{ d^{[i-1]}(s, t), d^{[i-1]}(s, v_i) + d^{[i-1]}(v_i, t) \right\}$$

Don't use vertex v_i

Take the min between the two

Use vertex v_i

Intermission

10 Tips to Succeed in EECS 376 (Part 3)

5

Reading Practice

3 Additional Problems



GeeksforGeeks



LeetCode

Practice **reading problems** carefully to understand their requirements. Don't underestimate the value of this skill—don't jump straight into solving the problem, but **ensuring you fully grasp what's being asked**.

6

Pattern Matching



“What **similar problems** have I seen before? What **methods or tricks** did I use, and can I apply them here?” You may just need to **replicate or modify** a known solution. Sometimes, all that's missing might just be an edge case!

Exchange Argument



The Exchange Step

- ▶ Notation:

$ALG = \{s_1, \dots, s_m\}$ = algorithm's (ordered) output; OPT = some *arbitrary* optimal solution

- ▶ If $ALG = OPT$, done. Otherwise,

- ▶ Let s_{diff} be the first element in ALG that **differs** from OPT (this means $s_1, \dots, s_{\text{diff}-1}$ agree with OPT)
- ▶ Modify OPT to agree with ALG up to s_{diff} (this involves modifying some $f \in OPT$ that is **not** in $s_1, \dots, s_{\text{diff}-1}$)
- ▶ Let's call this modified solution OPT'

- ▶ Now, we need to prove that

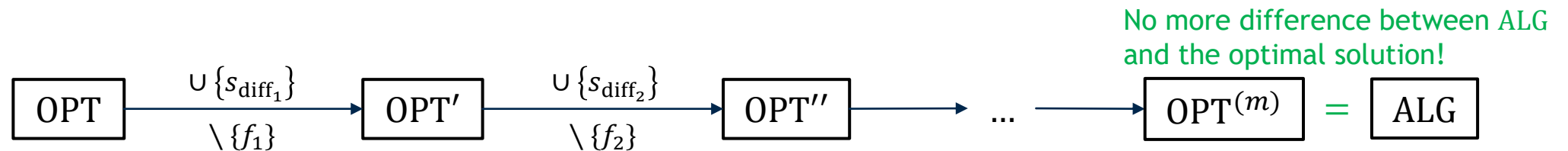
- ▶ OPT' is still a **valid** solution
- ▶ OPT' is still an **optimal** solution

Typically involve some
problem-specific/
algorithm-specific
observations



Induction and Exchange Argument

- ▶ **Goal:** Prove that ALG is an optimal solution
- ▶ **Exchange argument:** Show that we can transform **any optimal solution** into the **solution given by our algorithm** by **exchanging each piece** of it out one-by-one without increasing the **final cost**



- ▶ Induction framing for proving optimality
 - ▶ **Base case:** simplest form of the problem - often zero
 - ▶ **Inductive Hypothesis:** Assume that the first $k - 1$ choices of the greedy solution are part of **some** optimal solution
 - ▶ **Inductive step:** Show that the **first k choices (collectively, not just the k^{th} !)** are also part of **some** optimal solution (could be different from the one in IH)

Copying Homework Problems (#4)

- ▶ You are copying n homework problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.
- ▶ (a) Describe a greedy algorithm to minimize the number of pages you need.

Copying Homework Problems (#4)

- ▶ You are copying n homework problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.
- ▶ (a) Describe a greedy algorithm to minimize the number of pages you need.
- ▶ **Solution:**
 - ▶ Put as many problems as possible onto a single page.
 - ▶ Once a problem would exceed a line limit, move to the next page and repeat until all problems are copied.

Copying Homework Problems (#4)

- ▶ Put as many problems as possible onto a single page.
- ▶ Once a problem would exceed a line limit, move to the next page and repeat until all problems are copied.
- ▶ (b) Using an exchange argument, prove that the proposed greedy algorithm is correct. In other words, prove that the proposed algorithm uses the minimum number of pages, while satisfying the constraints given in the problem.

Proof: Copying Homework Problems (#4)

- ▶ Let $\text{ALG} = p_1, \dots, p_n$ be the nondecreasing sequence of page numbers on which our algorithm places the problems, where problem i appears on page p_i .
- ▶ Let $\text{OPT} = q_1, \dots, q_n$ be an arbitrary optimal solution, i.e., a non-decreasing sequence of page numbers where problem i appears on page q_i , and q_n is minimized.
- ▶ If $\text{ALG} = \text{OPT}$, then ALG is optimal, and there is nothing more to analyze.
- ▶ So suppose that $\text{ALG} \neq \text{OPT}$, and consider the first problem i that appears on a different page in ALG than in OPT; that is, $p_j = q_j$ for all $j < i$, but $p_i \neq q_i$. $\Rightarrow$ Then $p_i < q_i$ by the design of ALG.
- ▶ **Swap:** We construct a different optimal solution OPT' simply by replacing q_i with p_i such that it starts with $p_1, \dots, p_{i-1}, p_i$, i.e., it agrees with ALG on the first i problems.
- ▶ **Valid swap:** Notice that OPT' does not “overflow” page p_i , because ALG fits all the problems on page p_i that OPT' does, and possibly more. Moreover, no other page gets any additional problem(s) in OPT' versus OPT, so OPT' is indeed valid.
- ▶ **Optimal swap:** Note that OPT' does not increase the page number of the final problem n (or any problem), so it does not use more pages than OPT does. Since OPT is optimal, so is OPT' .
- ▶ Finally, by repeating the exchange step, we see that ALG agrees with some optimal solution for all n problems, hence ALG itself is optimal.

Counterexample

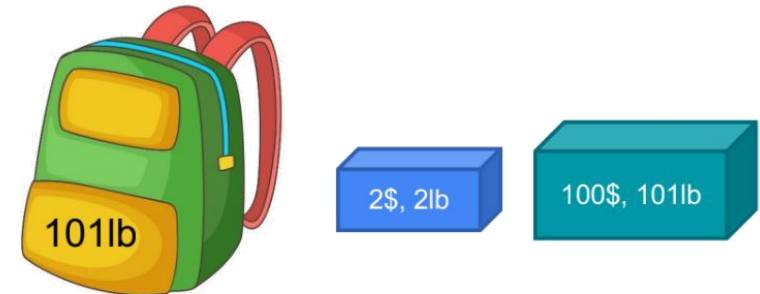


How to construct a counterexample?

- ▶ Generally, there is **no one-size-fit-all strategy** for disproving Greedy's optimality
- ▶ **Idea:** Want algorithm's locally optimal choices lead to globally suboptimal results
 - ▶ This typically involves situations where **an early choice** made by the algorithm **precludes** it from making significantly better choices later
- ▶ **Example:** Weight-Greedy Knapsack:

While there is still some unused knapsack capacity, add the item with smallest weight that could fit within the remaining knapsack capacity.

- ▶ Create an item with a small weight, ensuring it is **chosen early** by the algorithm
- ▶ This decision will **prevent** the algorithm from selecting a more valuable item later
- ▶ How does this happen? Set the knapsack capacity so that:
 - ▶ once the low-weight item is selected, there's insufficient room left for the high-value item



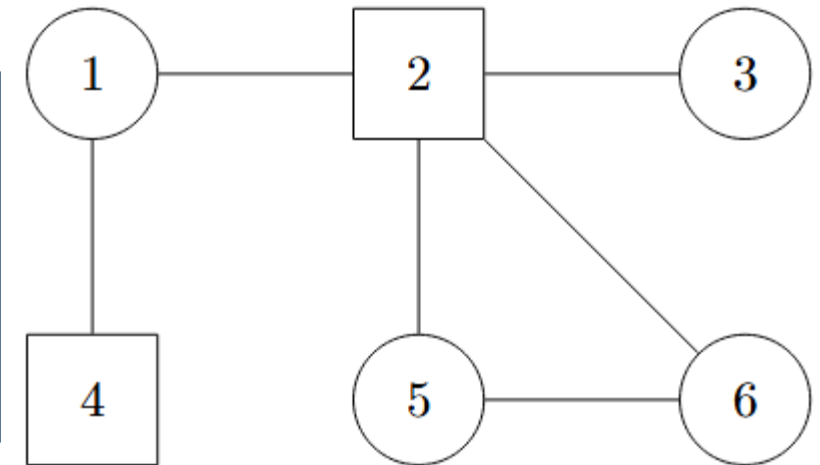
Dominating Set (WS #5)

A *dominating set* S in a graph G is a set of vertices for which every vertex of G either is in S , or is adjacent to some vertex in S .

We are interested in a *smallest* dominating set of a given graph, i.e., one that has the fewest possible vertices. (There may be more than one smallest dominating set.)

For example, the following graph has a smallest dominating set $S^* = \{2, 4\}$: every vertex other than 2 and 4 is adjacent to 2 or 4 (or both), and there is no dominating set consisting of a single vertex.

```
1: function GREEDYDS( $G$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has largest degree (i.e., the most neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all incident edges, from  $G$ 
7:   return  $S$ 
```



Dominating Set (WS #5)

Give a small graph G on which the algorithm might not return a *smallest* dominating set. Specifically, give a sequence of vertices that the algorithm might choose to make up its final output set, and give an optimal dominating set of G that is smaller than this output set.

```
1: function GREEDYDS( $G$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has largest degree (i.e., the most neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all incident edges, from  $G$ 
7:   return  $S$ 
```

Attendance! :O



Attendance!